

# FIAP

Enterprise Challenge 2026 • Sociedade 5.0

## SMART HAS — DIGITAL 360

Banco de Dados na Sociedade 5.0 | Fase 6 — Oracle PL/SQL + Java



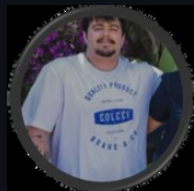
Eduardo Vasques

RM 556970



Otávio Souza

RM 550361



Enzo Paiva

RM 557632



Rafael Albuquerque

RM 559136



Guilherme Tibúrcio

RM 557500

GitHub: [github.com/DevRafonalde/digital360\\_flutter](https://github.com/DevRafonalde/digital360_flutter) Vídeo (YouTube): [inserir link]

São Paulo • Outubro de 2026

2

## O que a Fase 6 entrega

O motor de risco da AI Logistics saiu do app e passou a rodar no Oracle. App, API e rotinas automatizadas agora usam a mesma regra, e cada decisão fica registrada.

11

tabelas com PK, FK,  
CHECK e UNIQUE

4

functions PL/SQL

3

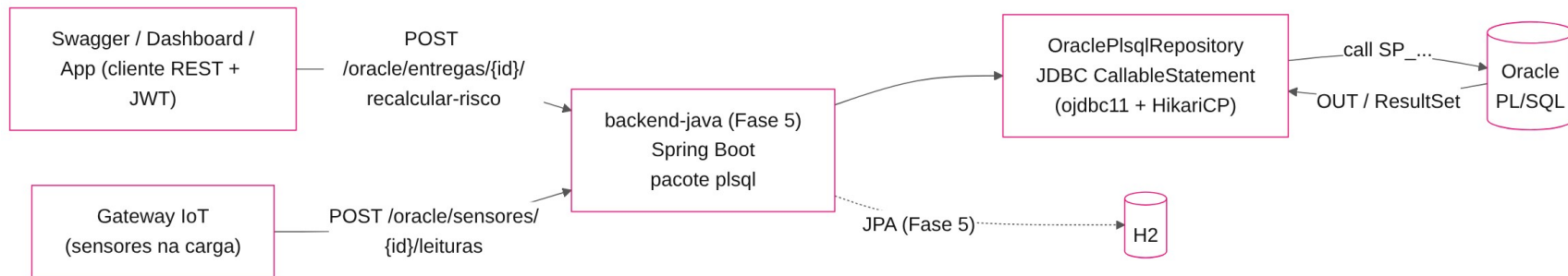
procedures com  
CURSOR, LOOP e  
EXCEPTION

6

endpoints /oracle no  
backend Java da Fase 5

**Novas funcionalidades:** alertas automáticos de sensores IoT, histórico de risco auditável, relatório de engajamento por usuário e modo híbrido no app (Logística via Oracle).

## 3 Arquitetura: REST → Java → JDBC → Oracle



### Clientes REST

Swagger, dashboard ou app chamam /oracle/\*\*; o gateway IoT envia leituras.

### backend-java (Fase 5)

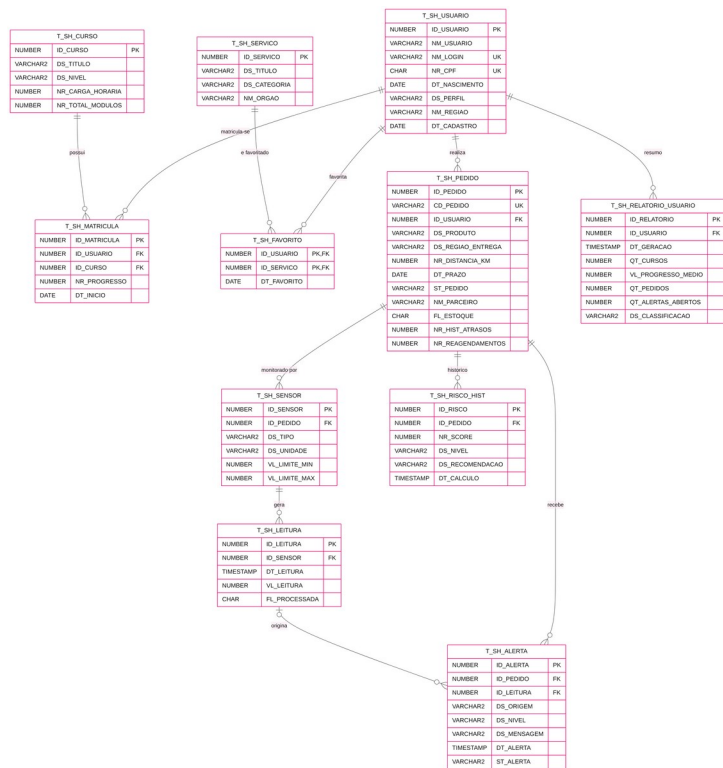
Novo pacote plsql: JWT, Swagger e erros ORA-200xx viram HTTP 404/400.

### Oracle PL/SQL

Regras de risco, alertas e relatório no banco, com transação controlada pela API.

## 4

# Modelo relacional (DER)



## Inclusão digital

USUARIO, CURSO, MATRICULA, SERVICO, FAVORITO

## Logística inteligente

PEDIDO, RISCO\_HIST, ALERTA, RELATORIO\_USUARIO

## IoT

SENSOR, LEITURA (telemetria de temperatura, umidade e vibração)

5

## Dados simulados importados

6

usuários fictícios

5

pedidos (1 crítico)

5

sensores IoT

19

leituras

6

leituras fora da faixa

Os pedidos 1 a 4 são os mesmos do app Flutter, então o app e o banco mostram os mesmos dados na demonstração.

| Sensor                  | Faixa segura | Leitura crítica   |
|-------------------------|--------------|-------------------|
| Temperatura (tinta 18L) | 5 a 40 °C    | 52,3 °C → CRITICO |
| Umidade (piso laminado) | 30 a 65 %    | 74 % → CRITICO    |
| Vibração (furadeira)    | 0 a 3 g      | 3,4 g → ALTO      |

## 6 Functions: indicadores e dados formatados

**FN\_CALCULA\_RISCO**

Indicador

Score 0–100: atrasos, reagendamentos, estoque, distância e status

**FN\_PROGRESSO\_MEDIO\_USUARIO**

Indicador

Média de progresso do usuário nas trilhas

**FN\_NIVEL\_RISCO**

Classificação

BAIXO / MEDIO / ALTO / CRITICO (DETERMINISTIC)

**FN\_RESUMO\_PEDIDO**

Dados formatados

Linha pronta para app, painel ou notificação

```
SELECT FN_RESUMO_PEDIDO(ID_PEDIDO) FROM T_SH_PEDIDO  
WHERE FN_NIVEL_RISCO(FN_CALCULA_RISCO(ID_PEDIDO)) IN ( 'ALTO', 'CRITICO' );
```

```
→ LM-2026-0002 | Tinta acrílica 18L | ATRASADO | Risco 100 (CRITICO) | Prazo 15/06/2026
```

## 7 Procedures: rotinas automatizadas no banco

### SP\_REGISTRAR\_ALERTAS\_SENSORES

Varre leituras pendentes e grava alerta ALTO ou CRITICO para cada leitura fora da faixa. Idempotente.

*CURSOR FOR UPDATE • LOOP • IF/ELSIF  
• WHERE CURRENT OF*

### SP\_RECALCULAR\_RISCO

Motor de risco: score, nível, recomendação, histórico e alerta sem duplicar. Acionada pela API Java.

*1 IN + 4 OUT • reusa functions •  
EXCEPTION*

### SP\_GERAR\_RELATORIO\_USUARIOS

Resumo de consumo por usuário: trilhas, progresso, pedidos, alertas e classificação.

*Cursor explícito • OPEN/FETCH/CLOSE •  
%ISOPEN*

## 8

## Integração Java acionando a procedure

```
POST /oracle/entregas/2/recalcular-risco

cs = con.prepareCall(
    "{call SP_RECALCULAR_RISCO(?,?,?, ?, ?)}");
cs.setLong(1, idPedido);
cs.registerOutParameter(2, NUMERIC);
cs.registerOutParameter(3, VARCHAR);
...
cs.execute();

{ "riscoScore": 100,
  "riscoNivel": "CRITICO" }
```

### Dentro do backend da Fase 5

Pacote plsqli no backend-java, com o mesmo login JWT e Swagger.

### Transação única

Leitura IoT + procedure de alertas: commit junto ou rollback junto.

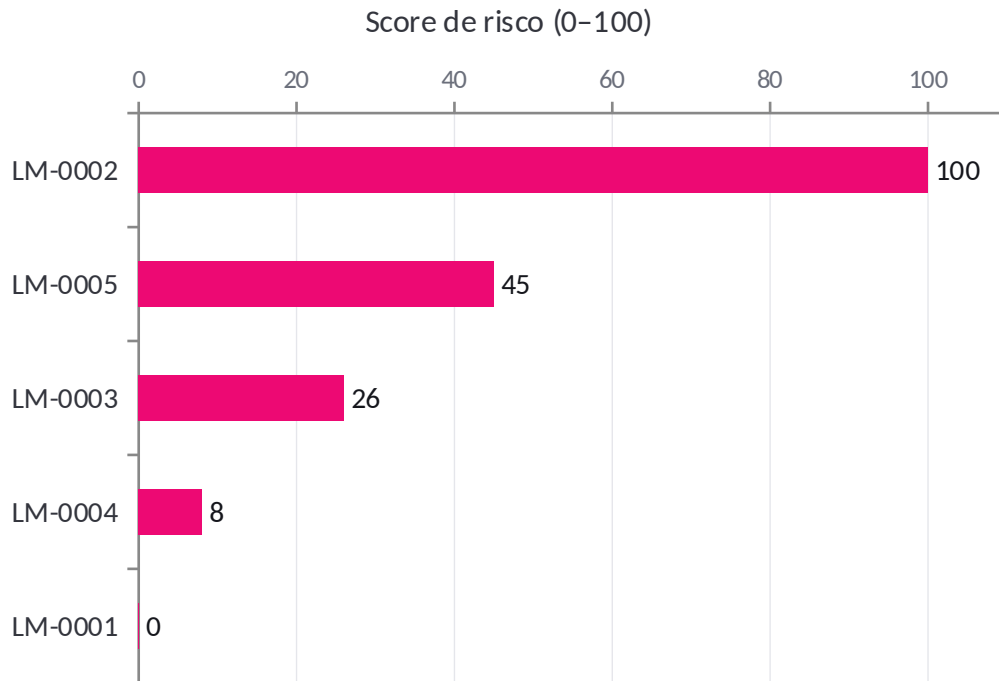
### Erros tratados

ORA-20001 → 404, ORA-20002 → 400, falha técnica → 503.



9

## Resultados: risco por pedido calculado no Oracle



6

alertas de sensor gerados

1

alerta de risco (só o pedido CRITICO)

6

usuários no relatório: 1  
ENGAJADO, 3 EM\_PROGRESSO, 2  
INICIANTE

# Valor gerado e próximos passos

## O que ganhamos

- Uma regra de risco, um só lugar
- Integridade garantida pelo banco
- Auditoria de cada decisão
- Alertas automáticos de IoT

## Próximos passos

- Tela de alertas Oracle no dashboard
- App consumindo /oracle/\*\* direto
- Job DBMS\_SCHEDULER para os alertas
- Testes com usuários reais

GitHub: [github.com/DevRafonalde/digital360\\_flutter](https://github.com/DevRafonalde/digital360_flutter) Vídeo: [inserir link]

**Obrigado!**